Prove Less, Repair Better: Automation-Friendly Program Repair for Small Models in Lean 4

Inesh Dheer Poojitha J Gracy Garg
Agents of Ultron — Agentic Software Engineering (CS6.403), IIIT Hyderabad

Abstract

Small, locally runnable language models rarely write valid Lean 4 proofs, yet they often write correct code. We test whether they produce more *end-to-end verified* repairs when they change only the implementation, shaping it so that a fixed, pre-registered portfolio of Lean automation proves the unchanged specification (condition C), than when they write the proofs themselves (B), or write proofs after the same automation has already been tried (BP). On repair tasks derived from VERINA, with natural bugs from other models and controlled mutants, budget-matched agents, hidden tests whose preconditions are proved in Lean, and a pre-registered analysis with the task family as the unit, we find that C verifies far more repairs than B for both primary models (Qwen3.5-2B: 27.7% of families vs 2.2%; Qwen3.5-4B: 33.0% vs 17.6%; Holm $p < 0.001$), and at least as many as BP (2B: +5.0 percentage points, $p_{\text{Holm}} = 0.012$; 4B: a tie), at about 43% fewer model tokens per verified repair than BP. The held-out split replicates C over B and shows a C/BP tie for both models. The gain comes from the automation more than from forbidding proofs: proof-writing agents that are given the same portfolio first close most of the gap. We also measure *why*: how often successes re-shape code for automation rather than revert the bug, whether models can re-shape correct but unprovable code, and which code shapes automation proves. Every step, from building the dataset to the final analysis, ran on one 48 GB MIG slice of an NVIDIA RTX PRO 6000 Blackwell GPU, with 4-bit models.

1 Introduction

Proof generation is the bottleneck of verified code generation. On VERINA [15], the best model in the published evaluation reached 72.6% code correctness but only 4.9% proof success. Small models, the ones developers can run privately and cheaply, are weaker still. Lean 4, however, ships strong automation (`simp`, `omega`, `decide`, `aesop`, `grind`) that closes many obligations without hand-written tactics, *provided the code has a suitable shape*.

We therefore move the proof burden from the model to the shape of the code. In condition C the model edits only the implementation, and a fixed, pre-registered tactic portfolio must prove the unchanged contract. The model never writes a tactic. We compare C against budget-matched proof-writing agents: B, which repairs code and proof from Lean feedback; BP, which gets the same portfolio first and writes a proof only if it fails; and A, a single-shot baseline.

Research questions. **RQ1** (confirmatory): at matched budget, does C reach a higher verified-repair rate than B and BP? **RQ2** (mechanism): do successes come from re-shaping code for automation or from reverting the bug, and can models re-shape correct but unprovable code (S vs S-plain)? **RQ3** (feedback): does C's structured hint beat a plain retry (C vs C-plain)? **RQ4** (code shapes): which implementation features predict that automation succeeds? **RQ5** (costs and scale): tokens, inference time, Lean time, patch size and integrity flags; 2B vs 4B; Qwen vs Gemma; a proof-specialist reference.

Contributions. (i) A budget-matched comparison of code-only repair against proof-writing repair for small models, with a B+P control that separates "do not write proofs" from "automation exists"; (ii) a mechanism analysis (shaping vs reverting, study S, and a code-shape catalogue); (iii) a buggy-VERINA dataset with natural and mutant bugs and hidden tests with Lean-proved preconditions; (iv) a fully pre-registered protocol, with a frozen configuration hash and a public deviation log. We do not claim novelty for compiler feedback, structured repair messages, automated tactics or code restructuring in isolation.

2 Related Work

Proof repair from compiler feedback. APRIL trains models to repair Lean proofs from compiler errors and goal states [6]; APOLLO combines Lean diagnostics, decomposition and automated solvers [12]. Both repair the *proof*; we leave the proof to fixed automation and repair the *code*. **Proof optimisation and joint planning.** ImProver and Lean Refactor rewrite existing proofs [2, 11]; P^3 plans programs and proofs jointly [10]; Vero evaluates frontier agents building verified repositories [16]. **Restructuring code for verification.** Dissect-and-Restore

transiently refactors Dafny code for SMT-based verification [1], and DafnyFix repairs Dafny implementations with single transformations [4]. Neither uses small models or Lean automation. **Verifier scaffolds and feedback.** A Lean verifier-guided scaffold documents specification hacking under verifier rewards [7]; FeedbackEval and related work compare feedback types for LLM repair [8, 5, 9]. **Benchmarks and their defects.** VERINA [15] and CLEVER [14] pair specifications with implementations. Two audits report VERINA specification issues [3, 13]; all of them were fixed upstream before our pinned commit (section 4).

3 Method

3.1 Tasks and bugs

VERINA has 189 Lean 4 tasks (pinned commit `66b3ce5`). A task is eligible if its reference compiles and passes its tests; a human proof is *not* required (only 46 of 189 ship one). Five tasks are excluded (four known defects and one with unrenderable tests), leaving 184 eligible: 104 basic and 80 advanced. Families are the statistical unit and are split at family level into pilot (20), main (114) and held-out (50).

Natural bugs are zero-shot implementations by Qwen3.5-0.8B and Qwen3.5-9B, which never act as repairers: at most four samples per generator and family, kept if they compile and fail at least one test. *Mutants* apply arithmetic, relational, boolean, constant, condition-negation, branch-swap, off-by-one and argument-swap operators, kept if they compile and fail a test, at most two per family. The pre-registered pool rule uses natural bugs if at least 60 `main` families carry one, and natural ∪ mutant otherwise. Natural bugs cover 33 of 114 `main` families, so the fallback pool applies; mutants are also reported separately. *Shaping instances* for study S are correct references that the portfolio alone cannot prove.

Hidden tests are generated inputs whose precondition is *proved in Lean*, with expected outputs from the reference (30 per family). They are never shown to a model; failures are reported only as a count.

3.2 Conditions

Table 1 lists the conditions. All share the same models, buggy starts, turn-0 information and budget per (instance, condition, model, seed): at most 8 generations, 48k tokens and 600 s of inference, whichever binds first, with at most 1,536 completion tokens per generation. C and C-plain sample two candidates per feedback round. Decoding uses temperature 0.7, top-p 0.95, hidden reasoning disabled, and seeds 11, 23 and 37.

3.3 Verification and the portfolio

Verification uses Lean 4.24.0 with the whole of Mathlib v4.24.0, loaded once per worker in our own verification server. A candidate is a *verified repair* only if it compiles, passes all visible and hidden tests, is proved by the allowed source, uses no axioms beyond `propext`, `Classical.choice` and `Quot.sound`, leaves the locked regions (specification, theorem statement, tests) unchanged, and passes static integrity checks. These reject `sorry`, `admit`, new axioms, `native_decide`, `implemented_by`, `unsafe`, `set_option`, macros, attributes, redefinitions of locked names, and any proof text in code-only conditions. Spec-copying, constant bodies, hard-coded tests and `choice` are flagged for audit.

The portfolio is frozen. It runs in a fixed order and stops early: Tier L (50k heartbeats) holds `decide`, `simp`, `simp_all`, and combinations with `omega` and `split`; Tier H (200k heartbeats) holds `grind`, `aesop`, and functional or structural induction closed by `simp_all` or `grind`. Heartbeats are the tactic budget; wall-clock guards only stop hangs. Infrastructure failures never charge the model's budget.

3.4 Analysis

The family score is the mean over seeds of "any instance of the family solved". RQ1 uses a one-sided paired sign-flip permutation test on family scores (100,000 permutations), Holm-corrected over the four primary tests (C vs B and C vs BP, for each primary model), plus exact McNemar on majority-solved families, 95% family-cluster bootstrap intervals (10,000 resamples) and Cohen's h. Secondary analyses are never pooled with the primary ones: **B_noauto** counts B's successes whose proof is a single call to automation as failures; **Bhammer** is compared with B and C; and the **B→C cross-evaluation** re-checks every B attempt whose code compiled but whose proof failed through C's code-only pipeline. The held-out split is analysed once and separately, as a replication.

4 Pre-registration, Freeze and Deviations

One environment for everything. Every step of the study ran on one 48 GB MIG slice (2g.48gb) of an NVIDIA RTX PRO 6000 Blackwell GPU, with an AMD Threadripper PRO 7975WX host: building the dataset (reference checks, hidden tests, natural bugs, mutants, shaping instances, splits), the probes, the pilot, the power simulation, every study run and the analysis. Models are the six SHA-256-verified 4-bit GGUF files, served by llama.cpp at a pinned commit (`162e2bb`) with hidden reasoning disabled.

Freeze. A 492-run pilot (20 families, 41 buggy instances, primary models, seed 11, all six conditions) calibrated the protocol, and a simulation from it estimated the

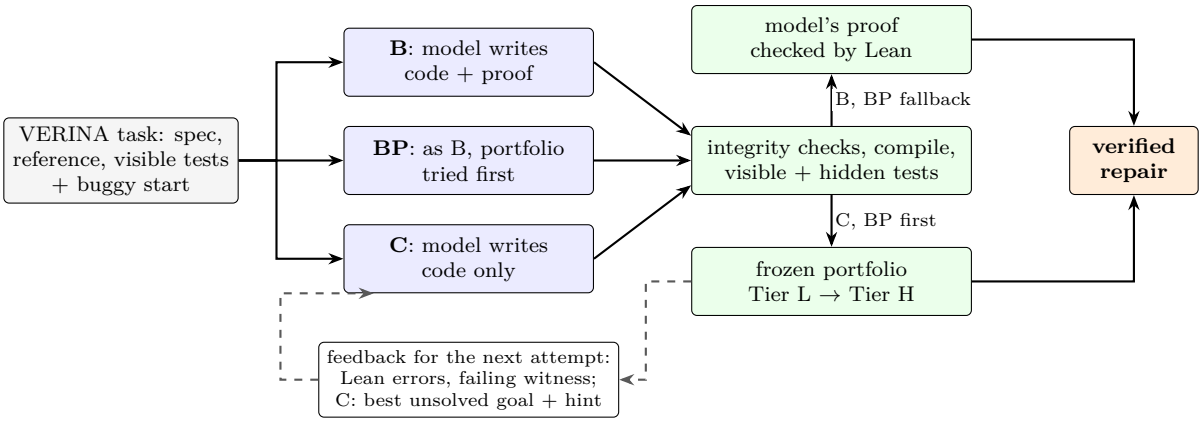


Figure 1: Method. Every condition gets the same buggy start, the same visible information and the same budget (8 generations, 48k tokens, 600 s of inference). A candidate must pass integrity checks, compile and pass every visible and hidden test before any proof is tried; hidden tests are never shown to the model. In B the model's own proof must check; in C the model writes no proof and a frozen two-tier portfolio of Lean automation must prove the unchanged specification; BP tries the portfolio first and falls back to the model's proof. A failed attempt returns feedback for the next one (dashed), in every condition except the single-shot baseline A.

Table 1: Conditions. The proof source is the only allowed one; everything else is shared.

ID	Model writes	Proof from	Feedback
T	—	portfolio	—
A	code + proof	model	none (N samples)
B	code + proof	model	Lean errors, witness, proof errors
BP	code + proof	portfolio, else model	as B
C	code only	portfolio	witness, best unsolved goal, structural hint
C-plain	code only	portfolio	generic retry
S / S-plain	code, from a correct ref.	portfolio	as C / as C-plain
Bhammer[†]	code + proof; prompt lists portfolio	model	as B

[†]Exploratory, pre-registered to run unconditionally; never pooled with B.

power of the primary tests (table 2). The full configuration (study parameters, portfolio, hints, prompts, power note) was then frozen under a SHA-256 digest (`3f2744e1...`, 19 files, 2026-11-05). The driver re-hashes it before every confirmatory run and refuses to run on any mismatch. Seven team decisions were fixed beforehand as rules and applied to the pilot numbers: the screening gate (portfolio coverage of eligible references 22.3%: basic 35.6%, advanced 5.0%); the bug-pool rule (33 of 114 `main` families carry a natural bug, against a minimum of 60); a 600 s inference cap, chosen because a 240 s cap was shown in an earlier calibration not to be condition-neutral; a C/BP tie to be reported with its power figures; Bhammer to run unconditionally; 4-bit models; and no new exclusions, because the reported VERINA specification issues [3, 13] were fixed upstream by an ancestor of our pinned commit.

An earlier freeze, prepared on other hardware, was superseded before any confirmatory result was analysed, so that every step would run in this one environment; the public deviation log records it.

Operational choices, all fixed before the second freeze:

Table 2: Pre-freeze power of the primary tests at the 114 available `main` families (500 simulated studies, sign-flip at $\alpha/4$), from the pilot's discordant-pair rates: point estimate and the pessimistic 80% Jeffreys bound, and the families needed for 80% power at the point estimate.

Comparison	Model	Point	Pessimistic	Families needed
C vs B	Qwen3.5-2B	1.000	0.662	34
C vs B	Qwen3.5-4B	0.996	0.128	57
C vs BP	Qwen3.5-2B	0.706	0.000	136
C vs BP	Qwen3.5-4B	0.082	0.000	not reachable

two jobs ran side by side with 12 concurrent requests each, which is where the slice saturates while keeping per-request decoding speed at the level the time cap was calibrated on. Jobs that ran beside the main job wrote to their own study folders, which the analysis reads together.

Post-freeze deviations. Three were logged after the runs finished; none changes a run, a verdict or a pre-registered rule. D-1: the energy meter read the NVML counter of a different physical GPU (NVML ignores the

CUDA device mask), and its Lean CPU estimate mixed the two concurrent jobs, so no energy or CO_2 figure is reported; per-run tokens and times are unaffected. D-2: two bugs in the audit sampler that made it crash and drop a column were fixed, without changing which runs it samples. D-3: the audit was carried out by Claude and is awaiting human confirmation.

5 Results

5.1 RQ1: verified-repair rate

Table 3 gives the four confirmatory tests on the 106 `main` families that carry a bug in the primary pool, and fig. 2 shows the rates for every condition. **C vs B** is large and significant for both primary models: Qwen3.5-2B 27.7% vs 2.2% (+25.5 pp, 95% CI [+18.2, +33.0]); Qwen3.5-4B 33.0% vs 17.6% (+15.4 pp, [+9.4, +21.7]); $p_{\text{Holm}} < 0.001$ for both. C repairs 28 families that B never repairs for 2B (B: 0 that C misses), and 19 against 1 for 4B. **C vs BP** is small. For 2B, C is ahead by +5.0 pp ([+1.6, +8.8], $p_{\text{Holm}} = 0.012$; 10 families only by C, 2 only by BP). For 4B the two are tied: 33.0% vs 32.7% (+0.3 pp, [−2.8, +3.5], $p_{\text{Holm}} = 0.50$; 3 vs 2 families). The pre-freeze power analysis predicted exactly this: C vs BP had 0.71 power for 2B and 0.08 for 4B at the available families (table 2), so the 4B tie is reported as a tie, not as evidence of no difference. Per-run rates (table 4) and outcome labels (table 6) tell the same story. No run in any condition was *verified but behaviourally wrong*, the outcome that would expose a weak specification. Most failures in the proof-writing conditions are submissions rejected by the static checks; they are mostly malformed rather than adversarial (lemmas placed in the helper region, a second definition of the function, or proof text in the code region), and they are far rarer in C and C-plain (1,690 and 1,586 rejected attempts) than in A, B and BP (7,187, 5,662 and 5,764).

5.2 RQ2: shaping, reverting and study S

We classify each success as a *revert* (identical to the reference, or within the pre-registered distance of a reference that the portfolio proves), *shaping* (the reference itself is *not* provable by the portfolio, so the verified code must have a different, automation-friendly shape) or other. Shaping is common: 42 of 150 C successes for 2B and 81 of 191 for 4B are shaping, against 72 and 56 reverts. Where the reference is provable by the portfolio, C and BP both succeed on most runs (4B: 89.4% and 91.1%; B: 36.6%). Where it is not, the rates fall for every condition, but C stays ahead of B (4B: 14.2% vs 6.0%; 2B: 7.4% vs 0.0%) and level with BP (12.3%). **Study S** asks the harder question directly: starting from a *correct* reference that the portfolio cannot prove, can a model re-shape it so that the portfolio can? Rarely, and the structured hint does not help: 3.7% (S) vs 4.1% (S-plain) of runs for 2B and 12.7% vs 12.7% for 4B (family-level differences −0.4 and 0.0 pp, $p = 0.81$ and 0.60). This suggests that shaping in the repair setting happens mostly when fixing the bug and re-shaping the code coincide, not as a skill of its own.

5.3 RQ3: structured hint vs plain retry

The structured hint (witness, best unsolved goal, structural hint) does not beat a plain retry. C vs C-plain: 2B 27.7% vs 28.9% (−1.3 pp, [−3.8, +1.3], $p = 0.89$), 4B 33.0% vs 32.7% (+0.3 pp, [−2.2, +2.8], $p = 0.50$); 3 vs 3 and 2 vs 1 discordant families. C-plain is slightly cheaper (4B: 9.1k vs 10.3k tokens and 58 vs 68 s of inference per run). The benefit of C comes from the code-only contract and the portfolio, not from the feedback text.

5.4 RQ4: which code shapes automation proves

Recursion is the dominant blocker. Among the 184 eligible references (condition T), the portfolio closes none of the 101 that use any recursion and 49.4% of the rest; `let rec` (0 of 77) and `match` (1.5% vs 33.9%) follow. The same holds for code the models write: among test-passing candidates from the code-only conditions, 0.7% of those with recursion are closed against 67.2% without, 10.3% with `match` against 62.6%, 29.7% with `a[i]!` indexing against 58.7%, and 23.7% with `Nat/Int` casts against 58.1%. Folds and list combinators cost little (−4 and −10 pp). Verified repairs are closed mostly by the cheapest entries: for C, `simp` closes 146 of 399, `unfold + omega` 90, `simp_all` 58, `simp+omega` 38 and `simp+grind` 36; the induction entries never close a repair. Automation-friendly code, in practice, means non-recursive code built from library combinators.

5.5 RQ5: costs, scale and the proof specialist

Cost. C is the cheapest route to a verified repair (fig. 3). For Qwen3.5-4B, C spends 37k model tokens per verified repair, against 65k for BP and 165k for B, and 68 s of inference per run against 162 s and 150 s; for 2B, 50k against 89k (BP) and 1.5M (B). C spends more Lean time per run (18 s vs 6 s for B with 4B) because the portfolio runs on every candidate. Truncating the generation budget to 2 or 4 shows that C's lead is already present at $N = 2$ (4B: 21.8% vs 6.8% of runs for B). Patch sizes (normalised edit distance from the buggy start, successes only) are similar across conditions (4B: C 0.67, BP 0.52, B 0.77). **Scale.** Going from 2B to 4B helps B most (2.2% → 17.6%) and C least (27.7% → 33.0%): the smaller model gains most from not writing proofs. **Second family.** Gemma-4-E4B (secondary) repeats the pattern: C 30.2% vs B 1.9% (+28.3 pp, $p < 0.001$) and C vs BP 30.2% vs 27.4% (+2.8 pp, [−3.8, +9.4], $p = 0.29$). **Proof specialist.** Goedel-Code-Prover-8B, run as a reference in A and B only, verifies 16.5% and 20.8% of runs, below Qwen3.5-4B

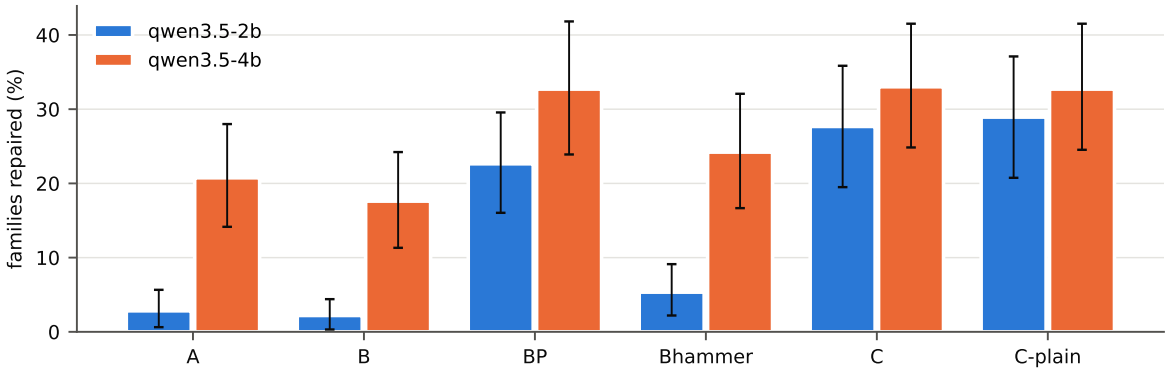


Figure 2: Verified-repair rate per condition on the `main` split, primary bug pool, as the pre-registered family score (mean over seeds of "any instance of the family solved"), with 95% family-cluster bootstrap intervals. Bhammer is exploratory.

Table 3: RQ1, confirmatory: paired family-level comparisons on the `main` split, primary bug pool (natural ∪ mutant). One-sided sign-flip test, Holm over the four primary tests; 95% family-cluster bootstrap CI; Cohen's h; families solved only by x / only by y.

Model	Comparison	Fam.	x	y	Diff. (pp) [95% CI]	p_{Holm}	h	only x/y
qwen3.5-2b	C vs B	106	27.7%	2.2%	+25.5 [+18.2, +33.0]	< 0.001	0.81	28/0
qwen3.5-2b	C vs BP	106	27.7%	22.6%	+5.0 [+1.6, +8.8]	0.012	0.12	10/2
qwen3.5-4b	C vs B	106	33.0%	17.6%	+15.4 [+9.4, +21.7]	< 0.001	0.36	19/1
qwen3.5-4b	C vs BP	106	33.0%	32.7%	+0.3 [−2.8, +3.5]	0.500	0.01	3/2

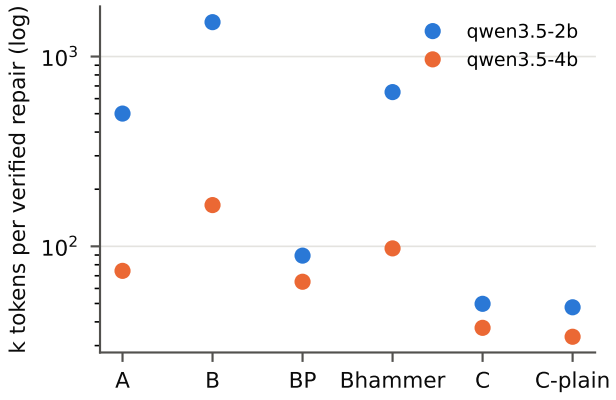


Figure 3: Model tokens spent per verified repair (log scale), `main` split, primary models. Missing points mean no verified repair in that cell.

in C (27.6%), which has half the parameters and no proof training. **Energy.** The GPU energy counter turned out to read a GPU other than ours, so no energy or CO_2 figure is reported (deviation D-1); tokens and inference seconds above are per-run measurements and are unaffected.

5.6 Secondary: B without automation, Bhammer and cross-evaluation

B without automation. Many of B's own proofs are a single automation call. Counting those as failures (B_noauto) lowers B from 17.6% to 12.3% for 4B (C vs B_noauto +20.8 pp); for 2B, B falls to 0.9%. **Bhammer**, which tells B the portfolio's tactics in the prompt, helps B a little (4B: 24.2% vs 17.6%, +6.6 pp, [+1.3, +11.9], $p = 0.011$; 2B: 5.3% vs 2.2%, $p = 0.057$) but stays below C (4B: −8.8 pp, $p = 0.001$; 2B: −22.3 pp, $p < 0.001$); 95 of its 131 successes for 4B are automation-only proofs. Knowing the tactics is not the same as having them run. **B→C cross-evaluation.** We re-verified every B attempt whose code compiled but whose proof failed through C's code-only pipeline. C's pipeline would have verified 30.8% of these attempts for 4B (797 of 2,588), 34.0% for 2B and 40.7% for Gemma, and would have rescued 17.3%, 16.8% and 23.1% of B's failed runs. None of these candidates passed the proof but failed the tests, so the cross-evaluation found no weak specification. In B, the code was often right and the proof was the bottleneck. **Audit.** The 10% stratified audit plus every flagged success (333 runs, 126 distinct programs) found no specification gaming. It was carried out by Claude and is awaiting human confirmation (deviation D-3). 27 runs decide the postcondition proposition directly or carry dead code, and are reported as unnatural but correct. The spec-copying

Table 4: Verified-repair rate per run on the `main` split (runs in brackets).

Model	A	B	BP	Bhammer	C	C-plain	S	S-plain
gemma-4-e4b-it		0.9% (231)	22.9% (231)		25.1% (231)			
goedel-code-prover-8b	16.5% (231)	20.8% (231)						
qwen3.5-2b	1.9% (693)	1.0% (693)	15.9% (693)	3.0% (693)	21.6% (693)	22.1% (693)	3.7% (267)	4.1% (267)
qwen3.5-4b	14.4% (693)	11.4% (693)	26.3% (693)	18.9% (693)	27.6% (693)	27.1% (693)	12.7% (267)	12.7% (267)

Table 5: Secondary and exploratory comparisons on `main` (uncorrected one-sided p). Never pooled with the primary estimates.

Model	Comparison	Fam.	x	y	Diff. (pp) [95% CI]	p	h	only x/y
gemma-4-e4b-it	C vs B	106	30.2%	1.9%	+28.3 [+19.8, +36.8]	< 0.001	0.89	30/0
gemma-4-e4b-it	C vs BP	106	30.2%	27.4%	+2.8 [−3.8, +9.4]	0.291	0.06	8/5
gemma-4-e4b-it	BP vs B	106	27.4%	1.9%	+25.5 [+17.9, +34.0]	< 0.001	0.83	27/0
gemma-4-e4b-it	C vs B_noauto	106	30.2%	0.0%	+30.2 [+21.7, +38.7]	< 0.001	1.16	32/0
qwen3.5-2b	C vs C-plain	106	27.7%	28.9%	−1.3 [−3.8, +1.3]	0.893	-0.03	3/3
qwen3.5-2b	C vs A	106	27.7%	2.8%	+24.8 [+17.6, +32.4]	< 0.001	0.77	28/0
qwen3.5-2b	BP vs B	106	22.6%	2.2%	+20.4 [+14.2, +27.4]	< 0.001	0.69	20/0
qwen3.5-2b	C vs B_noauto	106	27.7%	0.9%	+26.7 [+19.2, +34.6]	< 0.001	0.91	30/0
qwen3.5-2b	C vs Bhammer	106	27.7%	5.3%	+22.3 [+15.7, +29.2]	< 0.001	0.64	27/0
qwen3.5-2b	Bhammer vs B	106	5.3%	2.2%	+3.1 [0.0, +6.6]	0.057	0.17	3/2
qwen3.5-2b	S vs S-plain	89	3.7%	4.1%	−0.4 [−2.2, +1.1]	0.812	-0.02	0/1
qwen3.5-4b	C vs C-plain	106	33.0%	32.7%	+0.3 [−2.2, +2.8]	0.500	0.01	2/1
qwen3.5-4b	C vs A	106	33.0%	20.8%	+12.3 [+6.6, +18.2]	< 0.001	0.28	16/1
qwen3.5-4b	BP vs B	106	32.7%	17.6%	+15.1 [+9.4, +21.4]	< 0.001	0.35	18/1
qwen3.5-4b	C vs B_noauto	106	33.0%	12.3%	+20.8 [+14.2, +27.7]	< 0.001	0.51	24/1
qwen3.5-4b	C vs Bhammer	106	33.0%	24.2%	+8.8 [+3.5, +14.2]	< 0.001	0.20	12/2
qwen3.5-4b	Bhammer vs B	106	24.2%	17.6%	+6.6 [+1.3, +11.9]	0.011	0.16	11/3
qwen3.5-4b	S vs S-plain	89	12.7%	12.7%	0.0 [−2.6, +3.0]	0.602	0.00	2/0

flag fired on 239 of the audited runs, all of which restate an exact, executable postcondition.

5.7 Held-out replication

The held-out split (46 families with a bug in the primary pool) was run once and analysed separately (table 7). C vs B replicates for both models: 2B 20.3% vs 1.4% (+18.8 pp, [+9.4, +29.0]), 4B 29.7% vs 10.1% (+19.6 pp, [+10.1, +30.4]); $p_{\text{Holm}} = 0.001$ for both. C vs BP is a tie for both models (20.3% vs 20.3% and 29.7% vs 29.7%; $p_{\text{Holm}} = 1.00$), so the 2B advantage of C over BP on `main` does not replicate. No condition repaired any of the 10 held-out families with natural bugs.

6 Threats to Validity

Construct. A proof certifies only its specification. Every verified repair must also pass hidden tests, and "verified but behaviourally wrong" is reported separately as a weak-contract signal. Known-defective tasks are excluded. **Selection.** Portfolio coverage is reported over the full eligible pool; condition T is a screening and stratification variable only, never a filter on tasks. **Hand-tuning.** Prompts, portfolio and hints were tuned on pilot families only, then frozen; the held-out split was run once. **Specification gaming.** Static rejection, the axiom check, degeneracy flags and a 10% stratified hand audit plus every flagged success guard against it. **Internal.** Budgets and turn-0 information are equal; BP gives B the same automation first, Bhammer tells B the automation, B_noauto and the cross-evaluation test whether B is a strawman. **Statistical.** C vs BP is underpowered by the fixed size of VERINA (table 2); a tie there is reported as such, with cost per repair. **External.** VERINA functions are small; results are for 4-bit models and two model families, on one GPU model, and may not transfer to full precision or to repository-scale Lean. **Hardware.** Every step used one GPU slice, one host and one llama.cpp build, so no comparison mixes environments. The slice ran no job beyond our two study lanes. The physical GPU hosts other users' slices, which would have made per-condition energy unattributable even with a correct counter (deviation D-1); verdicts do not depend on it. The pilot is a calibration on its own split and is never pooled with the study.

7 Conclusion

Across 12,249 study runs on one GPU slice, letting automation carry the proof is the most effective and cheapest way for small models to produce end-to-end verified repairs.

Table 6: Outcome labels per run on `main`.

Model	Cond.	verified repair	correct unverified	integrity violation	unresolved	runs
gemma-4-e4b-it	B	2	88	103	38	231
gemma-4-e4b-it	BP	53	49	89	40	231
gemma-4-e4b-it	C	58	68	19	86	231
goedel-code-prover-8b	A	38	42	149	2	231
goedel-code-prover-8b	B	48	71	97	15	231
qwen3.5-2b	A	13	34	621	25	693
qwen3.5-2b	B	7	117	352	217	693
qwen3.5-2b	BP	110	55	312	216	693
qwen3.5-2b	Bhammer	21	175	267	230	693
qwen3.5-2b	C	150	86	168	289	693
qwen3.5-2b	C-plain	153	87	162	291	693
qwen3.5-2b	S	10	52	137	68	267
qwen3.5-2b	S-plain	11	53	129	74	267
qwen3.5-4b	A	100	156	395	42	693
qwen3.5-4b	B	79	132	408	74	693
qwen3.5-4b	BP	182	74	381	56	693
qwen3.5-4b	Bhammer	131	143	340	79	693
qwen3.5-4b	C	191	113	253	136	693
qwen3.5-4b	C-plain	188	135	215	155	693
qwen3.5-4b	S	34	45	116	72	267
qwen3.5-4b	S-plain	34	52	88	93	267

Table 7: Held-out replication (analysed once and separately; not confirmatory).

Model	Comparison	Fam.	x	y	Diff. (pp) [95% CI]	p_{Holm}	h	only x/y
qwen3.5-2b	C vs B	46	20.3%	1.4%	+18.8 [+9.4, +29.0]	< 0.001	0.69	10/0
qwen3.5-2b	C vs BP	46	20.3%	20.3%	0.0 [−4.3, +4.3]	1.000	0.00	2/1
qwen3.5-4b	C vs B	46	29.7%	10.1%	+19.6 [+10.1, +30.4]	< 0.001	0.50	11/0
qwen3.5-4b	C vs BP	46	29.7%	29.7%	0.0 [−4.3, +5.8]	1.000	0.00	1/0

C beats proof writing (B) decisively, on `main` and on the held-out split, for two Qwen sizes and for Gemma. Against a proof-writing agent that gets the same automation first (BP), C ties or wins narrowly, and the narrow win does not replicate on held-out. It is clearly cheaper: about 43% fewer tokens per verified repair and less than half the inference time with 4B. The structured hint adds nothing over a plain retry; recursion, `match` and array indexing are what defeat the automation; and models rarely re-shape correct but unprovable code. The practical lesson is to give small models a fixed automation portfolio and ask them only for code. Further gains will come from widening what automation can prove, especially recursion, rather than from better feedback text.

Artefacts. The harness, buggy-VERINA dataset with hidden tests, frozen configuration, deviation log and all run logs are in the Leanstein repository.

Table 8: Pilot (calibration split: 20 families, 41 buggy instances, seed 11; per-run rate, runs in brackets). Never pooled with the study.

Model	A	B	BP	Bhammer	C	C-plain
qwen3.5-2b	0.0% (41)	0.0% (41)	12.2% (41)	2.4% (41)	26.8% (41)	19.5% (41)
qwen3.5-4b	22.0% (41)	19.5% (41)	31.7% (41)	22.0% (41)	36.6% (41)	26.8% (41)

References

[1] Dissect-and-restore: AI-based code verification with transient refactoring, 2025. arXiv preprint.

[2] ImProver: Agent-based automated proof optimization. In *International Conference on Learning Representations (ICLR)*, 2025.

[3] Certified program synthesis with a multi-modal verifier, 2026. arXiv:2604.16584.

[4] DafnyFix: Single-transformation automated repair in Dafny. In *IEEE/ACM International Conference on Automated Software Engineering (ASE), APR track*, 2026.

[5] Effective LLM code refinement via property-oriented and structurally minimal feedback, 2026. arXiv preprint.

[6] Learning to repair Lean proofs from compiler feedback (APRIL), 2026. arXiv preprint.

[7] Verifier environments for LLM generation of verified programs and proofs. Master's thesis, Massachusetts Institute of Technology, June 2026.

[8] D. Dai et al. FeedbackEval: A benchmark for evaluating LLMs in feedback-driven code repair tasks, 2025. arXiv preprint, revised 2026.

[9] T. Guo. Evaluating symbolic feedback granularity for LLM-based program repair. In *IEEE/ACM International Conference on Automated Software Engineering (ASE), APR track*, 2026.

[10] Z. Li et al. P^3: Joint program-and-proof planning for verified code generation, 2026. arXiv preprint.

[11] J. Lu et al. Lean Refactor: Multi-objective controllable proof optimization via agentic strategy search. In *International Conference on Machine Learning (ICML)*, 2026.

[12] Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. APOLLO: Automated LLM and Lean collaboration for advanced formal reasoning, 2025. arXiv preprint.

[13] Sosso, Arora, and Spitters. Isomorphism is not enough: An agentic audit of verified code generation in Lean, 2026. arXiv:2605.23772.

[14] Amitayush Thakur et al. CLEVER: A curated benchmark for formally verified code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[15] Zhe Ye et al. VERINA: Benchmarking verifiable code generation. In *International Conference on Learning Representations (ICLR)*, 2026.

[16] Zhe Ye et al. Vero: Can AI agents build formally verified software repositories?, 2026. arXiv preprint.